

# Darren Wang

(412) 245-0809 | [dpnimo11@gmail.com](mailto:dpnimo11@gmail.com) | [darren-wang.dev](https://darren-wang.dev) | [github.com/DPnimo11](https://github.com/DPnimo11) | [linkedin.com/in/darren-wang-penn](https://linkedin.com/in/darren-wang-penn)

## EDUCATION

### University of Pennsylvania

*B.S.E. in Computer Science; GPA: 3.99/4.0*

May 2028

*Philadelphia, PA*

- **Relevant Coursework:** Operating Systems, Databases, Algorithms, Machine Learning, Computer Vision, Big Data Analytics, Scalable Cloud Computing, Computer Systems, Graph Neural Networks, Natural Language Processing.

## TECHNICAL SKILLS

**Languages:** Python, C, C++, Java, OCaml, SQL, JavaScript, TypeScript, HTML, CSS

**Frameworks & Tools:** React, Next.js, Tailwind CSS, Node.js, Express, PostgreSQL, PyTorch, scikit-learn, NumPy, pandas, Polars, Docker, AWS, Apache Spark

## EXPERIENCE

### Computational Social Science Lab, University of Pennsylvania

*Research Assistant – Software Engineering*

June 2026 – Present

*Philadelphia, PA*

- Merged a NOMAD optimization using spatial candidate search and temporal blocking, accelerating radius-based contact estimation by  $35.6\times$  (95.7s to 2.69s) on a 500-user benchmark while preserving identical output.
- Developed and deployed a Next.js and ECharts experimentation platform to evaluate 8 stop-detection algorithms on randomized synthetic GPS trajectories, featuring interactive parameter sweeps and synchronized geospatial and timeline playback for a NetMob submission.

### Penn Spark

*Software Developer – Atelic Travel Client Project*

Feb. 2026 – Present

*Philadelphia, PA*

- Rebuilt Atelic's customer-facing marketing site in React and Tailwind, translating product designs into responsive pages showcasing three core travel-planning workflows.

### University of Pittsburgh

*Research Intern*

Aug. 2023 – Present

*Pittsburgh, PA*

- First-authored a manuscript submitted to *Nature Communications* evaluating AlphaFold 3 robustness, generalization, and binding-affinity correlation across 2,000+ wild-type, mutant, and chimeric protein-peptide predictions.
- Developed AF3 evaluation pipelines for mutation generation and ligand-swapping cross-modeling, using DockQ/RMSD and interface pLDDT/PAE to test structural robustness and correlation with experimental  $\Delta\Delta G$ .

## PROJECTS

### PennOS | C, Operating Systems, File Systems

May 2026

- Built a user-level Unix-like operating system in C that multiplexes thread-backed processes through 100 ms preemption and provides an interactive shell with redirection, executable scripts, and foreground/background job control.
- Designed a weighted-credit scheduler that distributes CPU time across three priority classes at a 9:6:4 ratio while preserving round-robin fairness; implemented wait/signal semantics, zombie reaping, and orphan reparenting.
- Engineered a persistent FAT-style file system with memory-mapped allocation metadata and chained-block storage, adding reference-counted descriptor inheritance, permissions, and exclusive-writer coordination across processes.

### Anime Recommendation Query Engine | React, Node.js, PostgreSQL, AWS RDS

May 2026

- Built a React/Express recommendation and analytics app over 31M+ user-anime interactions, backed by PostgreSQL on AWS RDS with search, filtering, and co-viewer queries.
- Reduced representative recommendation latency from  $\sim 35$ s to  $\sim 50$ ms by profiling slow joins and applying indexes, materialized views, query restructuring, and Express API caching.

### Distributed Social Graph Engine | Apache Spark, AWS EMR, DynamoDB

Dec. 2025

- Implemented weighted graph adsorption on Spark/EMR over friendship, interest, and article-engagement data from DynamoDB, materializing personalized news recommendations through iterative label propagation.
- Decoupled cluster computation from interactive requests by submitting jobs through Apache Livy, polling completion asynchronously, and refreshing feeds after Spark persisted recommendation results.

### LeetLoop | Next.js, TypeScript, Supabase, PostgreSQL

July 2026

- Designed a deterministic scheduler that separates ideal spaced-repetition dates from capacity-constrained daily plans, balances review collisions without leapfrogging older work, and preserves assignments across replans.
- Implemented optimistic client state with serialized, diff-based Supabase writes and per-user PostgreSQL row-level security, keeping interactions responsive while preserving mutation order and account isolation.